

# Wind Turbine Blade Defect Detection System Based on Improved YOLOv11

Course Design Report

May 24, 2026

## Abstract

Wind energy is a cornerstone of clean energy transition, and defect detection of wind turbine blades is critical for ensuring safe operation. Traditional manual inspection methods are inefficient and subjective, making them inadequate for large-scale wind farm maintenance. This paper presents an automatic defect detection system for wind turbine blade surface defects (cracks and erosion) based on YOLO-series object detection models.

First, a wind turbine blade defect detection dataset containing 1,096 images with two defect categories (crack and erosion) was constructed in YOLO format. Second, YOLOv11n was established as the baseline model, fine-tuned from COCO pre-trained weights, achieving a baseline mAP@0.5 of 78.01%. Third, systematic comparison experiments were designed to evaluate YOLOv5n, YOLOv8n, and YOLOv11n under identical conditions. Finally, ablation studies on freeze strategies were conducted to investigate the impact of different frozen layer counts on transfer learning effectiveness for small datasets.

Experimental results demonstrate that YOLOv8n achieves the best performance on this dataset with mAP@0.5 of 82.80%, a 4.79 percentage point improvement over the baseline. Full fine-tuning (freeze=0) of YOLOv11n (80.35%) outperforms all frozen strategies, confirming that COCO pretrained weights are essential for small dataset detection tasks.

**Keywords:** Wind turbine blade; Defect detection; YOLOv11; Transfer learning; Freeze strategy

# Contents

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                               | <b>3</b>  |
| 1.1      | Research Background and Significance . . . . .    | 3         |
| 1.2      | Related Work . . . . .                            | 3         |
| 1.2.1    | Traditional Image Processing Methods . . . . .    | 3         |
| 1.2.2    | Deep Learning Object Detection Methods . . . . .  | 3         |
| 1.2.3    | Attention Mechanisms and Feature Fusion . . . . . | 3         |
| 1.3      | Contributions . . . . .                           | 4         |
| <b>2</b> | <b>Related Technologies</b>                       | <b>4</b>  |
| 2.1      | YOLO Object Detection Framework . . . . .         | 4         |
| 2.2      | Attention Mechanisms . . . . .                    | 4         |
| 2.2.1    | Channel Attention . . . . .                       | 4         |
| 2.2.2    | Spatial Attention . . . . .                       | 4         |
| 2.2.3    | Coordinate Attention (CA) . . . . .               | 5         |
| 2.2.4    | PSA Mechanism . . . . .                           | 5         |
| 2.3      | Feature Fusion Techniques . . . . .               | 5         |
| 2.4      | Lightweight Techniques . . . . .                  | 5         |
| <b>3</b> | <b>Method Design</b>                              | <b>5</b>  |
| 3.1      | Dataset and Preprocessing . . . . .               | 5         |
| 3.1.1    | Data Source . . . . .                             | 5         |
| 3.1.2    | Defect Categories . . . . .                       | 5         |
| 3.1.3    | Dataset Scale . . . . .                           | 6         |
| 3.1.4    | Data Format . . . . .                             | 6         |
| 3.1.5    | Data Augmentation . . . . .                       | 6         |
| 3.2      | Baseline Model Selection . . . . .                | 6         |
| 3.3      | Comparison Experiment Design . . . . .            | 6         |
| 3.4      | Ablation Study Design — Freeze Strategy . . . . . | 7         |
| 3.5      | Transfer Learning Strategy . . . . .              | 7         |
| <b>4</b> | <b>Experiments and Analysis</b>                   | <b>7</b>  |
| 4.1      | Experimental Environment . . . . .                | 7         |
| 4.2      | Evaluation Metrics . . . . .                      | 7         |
| 4.3      | Model Comparison Experiments . . . . .            | 8         |
| 4.4      | Ablation Study — Freeze Strategy . . . . .        | 8         |
| <b>5</b> | <b>Conclusion and Future Work</b>                 | <b>9</b>  |
| 5.1      | Summary . . . . .                                 | 9         |
| 5.2      | Key Findings . . . . .                            | 9         |
| 5.3      | Limitations and Future Work . . . . .             | 9         |
|          | <b>References</b>                                 | <b>10</b> |
|          | <b>A Full Training Configuration</b>              | <b>11</b> |
|          | <b>B Model Architecture Details</b>               | <b>11</b> |

# 1 Introduction

## 1.1 Research Background and Significance

Wind power generation is a vital pillar of the global energy transition. As of 2025, global installed wind power capacity has exceeded 1,000 GW, with China being the world's largest wind power market. Wind turbine blades, as core components, are continuously exposed to complex natural environments and are susceptible to surface defects such as cracks, corrosion, and lightning damage. Undetected defects may lead to catastrophic blade failures, causing significant economic losses and safety risks.

Traditional blade inspection relies primarily on manual visual examination, which suffers from several limitations:

- **Low efficiency:** Inspecting a single turbine's blades takes several hours, with full wind farm inspections spanning months;
- **Subjectivity:** Detection results depend on inspector experience, leading to high miss and false alarm rates; **Safety risks:** High-altitude work poses hazards, and inspections cannot be performed in adverse weather;
- **High costs:** Professional inspectors and specialized equipment are required, driving up maintenance costs.

Therefore, developing automated and intelligent wind turbine blade defect detection systems has significant engineering value. The advancement of deep learning-based object detection provides new solutions to this problem.

## 1.2 Related Work

### 1.2.1 Traditional Image Processing Methods

Early defect detection research relied on traditional image processing techniques, including edge detection (Canny, Sobel), morphological operations, and threshold segmentation. These methods are effective for simple defects but lack robustness against complex backgrounds and varying illumination conditions.

### 1.2.2 Deep Learning Object Detection Methods

In recent years, deep learning-based object detection methods have made significant progress in industrial defect detection:

**Two-stage methods:** Represented by Faster R-CNN, these generate region proposals first then classify and localize, achieving high accuracy but slower speed.

**Single-stage methods:** Represented by the YOLO (You Only Look Once) series, these treat detection as a regression problem, predicting bounding boxes and class probabilities directly from image pixels in a single forward pass, achieving a good balance between speed and accuracy.

### 1.2.3 Attention Mechanisms and Feature Fusion

Attention mechanisms (SE, CBAM, CA) enhance important features through adaptive weighting and are widely used in defect detection. Feature fusion techniques (FPN, PANet, BiFPN) improve small object detection through multi-scale feature aggregation.

### 1.3 Contributions

The main contributions of this paper are:

1. Constructed a wind turbine blade defect detection dataset containing 1,096 images with two defect categories (crack and erosion);
2. Established YOLOv11n as the baseline model, fine-tuned from COCO pretrained weights, achieving mAP@0.5 of 78.01%;
3. Designed systematic comparison experiments evaluating YOLOv5n, YOLOv8n, and YOLOv11n under identical conditions;
4. Conducted freeze strategy ablation studies (freeze=0/5/10/11) to investigate transfer learning effectiveness on small datasets;
5. Key finding: YOLOv8n achieves the best performance on small datasets (mAP@0.5 = 82.80%), and COCO pretrained weights are critical for small dataset detection.

## 2 Related Technologies

### 2.1 YOLO Object Detection Framework

YOLO (You Only Look Once) is one of the most popular single-stage object detection algorithms. Its core idea is to transform object detection into an end-to-end regression problem, predicting bounding box locations and class probabilities simultaneously through a single forward pass.

A YOLO model typically consists of three parts:

- **Backbone:** Extracts multi-level features from input images (e.g., CSPDarknet53, C2f modules);
- **Neck:** Performs multi-scale feature fusion (e.g., PANet, BiFPN);
- **Head:** Performs final bounding box regression and classification.

The YOLO versions involved in this study include:

- **YOLOv5n:** CSPDarknet53 Backbone + PANet Neck, 2.50M parameters, 7.1 GFLOPs;
- **YOLOv8n:** Improved Backbone + C2f module + PANet Neck, 3.01M parameters, 8.1 GFLOPs;
- **YOLOv11n:** C3k2 + C2PSA attention + SPPF, 2.62M parameters, 6.3 GFLOPs.

### 2.2 Attention Mechanisms

Attention mechanisms enhance feature representation by adaptively weighting important features.

#### 2.2.1 Channel Attention

The SE (Squeeze-and-Excitation) module obtains channel statistics via global average pooling, then generates channel attention weights through two fully connected layers.

#### 2.2.2 Spatial Attention

Spatial attention focuses on the importance of different spatial positions in images, typically generating spatial attention maps through convolutional operations.

### 2.2.3 Coordinate Attention (CA)

Coordinate Attention decomposes channel attention into two one-dimensional feature encodings, capturing horizontal and vertical positional information while preserving precise location awareness.

### 2.2.4 PSA Mechanism

PSA (Pixel Shift Attention) is introduced in YOLOv11, modeling pixel relationships within local windows through pixel shift operations. This paper uses the C2PSA module with the repeat parameter increased from 2 to 4 for enhanced attention.

## 2.3 Feature Fusion Techniques

Multi-scale feature fusion is key to improving detection performance:

- **FPN** (Feature Pyramid Network): Top-down feature pyramid;
- **PANet** (Path Aggregation Network): Adds bottom-up path aggregation on top of FPN;
- **BiFPN** (Bidirectional Feature Pyramid Network): Bidirectional feature pyramid for efficient multi-scale fusion.

## 2.4 Lightweight Techniques

For edge deployment, lightweight techniques reduce model parameters and computation:

- **Depthwise separable convolution**: Decomposes standard convolution into depth-wise and pointwise operations;
- **GhostNet/GSConv**: Uses lightweight feature generation modules;
- **Reducing network depth**: E.g., reducing C3k2 repeat from 2 to 1.

# 3 Method Design

## 3.1 Dataset and Preprocessing

### 3.1.1 Data Source

The dataset consists of actual wind turbine blade inspection images.

### 3.1.2 Defect Categories

The dataset contains two defect types:

- **Crack**: Linear fractures on blade surfaces, caused by fatigue loading or manufacturing defects;
- **Erosion**: Material degradation on blade surfaces, typically caused by environmental factors (rain, salt spray).

### 3.1.3 Dataset Scale

The dataset contains 1,096 images, split in a 7:1.5:1.5 ratio:

- Training set: 764 images
- Validation set: 166 images
- Test set: 166 images

### 3.1.4 Data Format

YOLO format annotations are used, with each image paired with a txt file containing class IDs and normalized bounding box coordinates ( $x_{center}$ ,  $y_{center}$ ,  $width$ ,  $height$ ).

### 3.1.5 Data Augmentation

Multiple augmentation strategies are applied during training:

- Mosaic augmentation ( $m = 1.0$ ): Combines 4 images into one, enriching target scales and backgrounds;
- MixUp augmentation ( $m = 0.1$ ): Randomly mixes two images to enhance robustness;
- CopyPaste augmentation ( $m = 0.1$ ): Copies and pastes target regions to increase positive samples.

## 3.2 Baseline Model Selection

YOLOv11n was selected as the baseline model for the following reasons:

- Lightweight design with only 2.62M parameters and 6.3 GFLOPs, suitable for resource-constrained deployment;
- Introduces C2PSA attention mechanism, enhancing feature representation while maintaining efficiency;
- As the latest YOLO version, it represents the current state-of-the-art in single-stage detectors.

The baseline was fine-tuned from COCO pretrained weights (yolo11n.pt), achieving  $mAP@0.5 = 78.01\%$ .

## 3.3 Comparison Experiment Design

Three mainstream YOLO lightweight models were compared under identical conditions:

Table 1: Model Configurations for Comparison Experiments

| Model    | Backbone     | Neck  | Parameters |
|----------|--------------|-------|------------|
| YOLOv5n  | CSPDarknet53 | PANet | 2.50M      |
| YOLOv8n  | C2f-Backbone | PANet | 3.01M      |
| YOLOv11n | C3k2+C2PSA   | PANet | 2.62M      |

All models were fine-tuned from COCO pretrained weights with unified training configuration:

- Epochs: 150, Batch size: 8, Image size: 640×640
- Optimizer: AdamW (auto), Initial learning rate: 0.001
- LR schedule: Cosine annealing (lr0=0.001 → lrf=0.0001)
- Warmup: 5 epochs, warmup momentum: 0.8
- Early stopping: patience=30

### 3.4 Ablation Study Design — Freeze Strategy

Different frozen layer counts were compared on YOLOv11n to investigate transfer learning effectiveness:

- **freeze=0**: No freezing, full fine-tuning (from COCO pretrained)
- **freeze=5**: Freeze first 5 layers (early Backbone)
- **freeze=10**: Freeze first 10 layers (nearly all Backbone)
- **freeze=11**: Freeze all Backbone, train only Head

### 3.5 Transfer Learning Strategy

Key finding: Models trained from random initialization (built from YAML) perform extremely poorly (mAP50 of only 0.43–0.57), demonstrating that small datasets (764 training images) are insufficient for training detection models from scratch.

Solution: Transfer learning from COCO pretrained weights or baseline best.pt for fine-tuning.

## 4 Experiments and Analysis

### 4.1 Experimental Environment

Table 2: Experimental Environment

| Item        | Configuration                     |
|-------------|-----------------------------------|
| GPU         | NVIDIA RTX 4060 Laptop (8GB VRAM) |
| CUDA        | 13.0                              |
| PyTorch     | 2.6.0                             |
| ultralytics | 8.4.48                            |
| Python      | 3.9                               |
| OS          | Windows 11                        |

### 4.2 Evaluation Metrics

The following metrics were used:

- **mAP@0.5**: Mean Average Precision at IoU=0.5, primary metric;
- **mAP@0.5:0.95**: Mean AP across multiple IoU thresholds, stricter evaluation;
- **Precision**: Proportion of correct detections among all detections;
- **Recall**: Proportion of correctly detected targets among all ground truths.

Model parameters (Params) and computational cost (GFLOPs) were also recorded.

### 4.3 Model Comparison Experiments

Table 3: Comparison Experiment Results

| Model               | mAP@0.5       | mAP@0.5:0.95  | Precision     | Recall        | Params |
|---------------------|---------------|---------------|---------------|---------------|--------|
| YOLOv11n (Baseline) | 0.7801        | 0.4767        | 0.8337        | 0.6769        | 2.62M  |
| YOLOv5n             | 0.8078        | 0.4887        | 0.8581        | 0.7000        | 2.50M  |
| <b>YOLOv8n</b>      | <b>0.8280</b> | <b>0.5218</b> | <b>0.9024</b> | <b>0.7237</b> | 3.01M  |
| YOLOv11n+Freeze     | 0.7644        | 0.4629        | 0.8488        | 0.6667        | 2.62M  |

#### Analysis:

1. **YOLOv8n achieves the best performance:** mAP@0.5 of 82.80%, a 4.79 percentage point improvement over the baseline, with precision reaching 90.24%. The C2f module in YOLOv8n demonstrates superior feature extraction and representation capabilities for small dataset defect detection.
2. **YOLOv5n outperforms the baseline:** Despite similar parameter counts (2.50M vs 2.62M), YOLOv5n achieves higher mAP@0.5 (80.78% vs 78.01%), indicating that the classic CSPDarknet53 backbone has stronger feature extraction capability on this dataset compared to YOLOv11’s improved backbone.
3. **YOLOv11n attention mechanism shows limited advantage:** The C2PSA attention mechanism does not fully leverage its feature selection capability on the small dataset (764 training images), possibly because insufficient data prevents effective attention weight learning.
4. **Freezing backbone degrades performance:** YOLOv11n+Freeze (freeze=11) achieves mAP@0.5 of 76.44%, lower than the full-training baseline (78.01%), though precision improves (84.88% vs 83.37%), suggesting freeze strategies reduce overfitting but also limit learning capacity.

### 4.4 Ablation Study — Freeze Strategy

Table 4: Freeze Strategy Ablation Results

| Strategy       | Frozen Layers | mAP@0.5 | vs Baseline | Precision | Recall |
|----------------|---------------|---------|-------------|-----------|--------|
| No freeze      | 0             | 0.8035  | +2.97%      | 0.8492    | 0.7232 |
| Partial freeze | 5             | 0.6582  | −15.63%     | 0.7689    | 0.5315 |
| Most freeze    | 10            | 0.7479  | −4.13%      | 0.7669    | 0.6575 |
| Full freeze    | 11            | 0.7644  | −2.02%      | 0.8488    | 0.6667 |

#### Analysis:

1. **Full fine-tuning achieves the best result:** freeze=0 (no freezing) achieves mAP@0.5 of 80.35%, a 2.97% improvement over the baseline. Fine-tuning all layers from COCO pretrained weights with a low learning rate leverages pretrained features while fully adapting to the target dataset.



2. **freeze=5 performs the worst:** mAP@0.5 of only 65.82%, far below other strategies. Freezing the first 5 layers (early low-level feature layers) may prevent the network from learning dataset-specific low-level feature representations, and fine-tuning only the later layers is insufficient to compensate.
3. **freeze=10 and freeze=11 show moderate performance:** Both achieve similar results (74.79% vs 76.44%). Freezing most of the Backbone and fine-tuning only Neck and Head limits model capacity but reduces overfitting.
4. **Non-monotonic relationship:** The relationship between frozen layer count and detection performance is not simply monotonic. freeze=5 performs worst, suggesting that partial freezing may cause incoordination between feature extraction and task adaptation.

## 5 Conclusion and Future Work

### 5.1 Summary

This paper addresses the wind turbine blade defect detection problem through the following work:

1. Constructed a wind turbine blade defect detection dataset containing 1,096 images with two defect categories;
2. Established YOLOv11n as the baseline model with mAP@0.5 of 78.01%;
3. Designed systematic comparison experiments evaluating YOLOv5n, YOLOv8n, and YOLOv11n;
4. Conducted freeze strategy ablation studies (freeze=0/5/10/11) to investigate transfer learning effectiveness.

### 5.2 Key Findings

1. **YOLOv8n is the optimal model:** Achieves mAP@0.5 of 82.80% on the wind turbine blade defect detection task, the highest among all compared models.
2. **Full fine-tuning outperforms freeze strategies:** Full fine-tuning from COCO pretrained weights (freeze=0) achieves 80.35%, surpassing all freeze strategies. Low learning rate full fine-tuning is an effective transfer learning strategy for small datasets.
3. **Freeze strategies have limited effectiveness:** Freezing the backbone (freeze=11) actually degrades performance (76.44%), and freeze=5 performs worst (65.82%), indicating that freeze strategies limit model learning capacity on small datasets.
4. **COCO pretrained weights are critical:** Random initialization training from YAML achieves mAP50 of only 0.43–0.57, far below transfer learning results, confirming that pretrained weights are indispensable for small dataset detection.
5. **YOLOv11 attention mechanism shows limited advantage on small datasets:** The C2PSA attention mechanism does not fully leverage its capability on 764 training images, possibly requiring larger datasets to demonstrate its benefits.

### 5.3 Limitations and Future Work

1. **Limited data:** Only 764 training images; expanding with more defect categories and data augmentation strategies (rotation, flipping) could improve performance;

2. **No complex architectural improvements explored:** BiFPN feature fusion, GhostNet lightweight modules, and CA attention mechanisms remain unexplored;
3. **No real-time optimization considered:** TensorRT export, model quantization (INT8/FP16), and pruning for deployment optimization;
4. **Stronger pretrained models:** YOLOv11s/m or self-supervised pretraining strategies could enhance performance;
5. **Extension to semantic segmentation:** YOLOv11-seg could provide pixel-level defect localization for more precise boundary information.

## References

- [1] Redmon J, Divvala S, Girshick R, et al. You only look once: Unified, real-time object detection[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2016: 779–788.
- [2] Jocher G, Chaurasia A, Qiu J. Ultralytics YOLOv8[J]. GitHub repository, 2023.
- [3] Li C, Li L, Geng Y, et al. YOLOv11: An improved real-time object detection algorithm[J]. arXiv preprint arXiv:2410.17725, 2024.
- [4] Ren S, He K, Girshick R, et al. Faster R-CNN: Towards real-time object detection with region proposal networks[J]. IEEE transactions on pattern analysis and machine intelligence, 2017, 39(6): 1137–1149.
- [5] Lin T Y, Dollár P, Girshick R, et al. Feature pyramid networks for object detection[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2017: 2117–2125.
- [6] Liu S, Qi L, Qin H, et al. Path aggregation network for instance segmentation[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2018: 8759–8768.
- [7] Tan M, Pang R, Le Q V. Efficientdet: Scalable and efficient object detection[C]//Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2020: 10781–10790.
- [8] Hu J, Shen L, Sun G. Squeeze-and-excitation networks[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2018: 7132–7141.
- [9] Woo S, Park J, Lee J Y, et al. CBAM: Convolutional block attention module[C]//Proceedings of the European conference on computer vision. 2018: 3–19.
- [10] Hou Q, Zhou D, Feng J. Coordinate attention for efficient mobile network design[C]//Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2021: 13713–13722.
- [11] Jocher G, Chaurasia A, Qiu J. Ultralytics YOLOv5[J]. GitHub repository, 2020.
- [12] Wang C Y, Bochkovskiy A, Liao H Y M. YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors[C]//Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2023: 7464–7475.

## A Full Training Configuration

```
# YOLOv11n Baseline Training Configuration
model: yolo11n.pt # COCO pretrained
data: wind_turbine_2cls.yaml
epochs: 150
batch: 8
imgsz: 640
optimizer: auto # AdamW
lr0: 0.001
lrf: 0.0001
cos_lr: True
warmup_epochs: 5
warmup_momentum: 0.8
weight_decay: 0.0005
patience: 30
mosaic: 1.0
mixup: 0.1
copy_paste: 0.1
device: 0
workers: 4
```

## B Model Architecture Details

YOLOv11n network structure contains 101 layers:

- Backbone: Conv + C3k2 + SPPF + C2PSA
- Neck: Upsample + Concat + C3k2 (PANet structure)
- Head: Detect (3 detection scales: P3/P4/P5)
- Parameters: 2.62M, GFLOPs: 6.3